

LAPORAN TUGAS 6

Pemrograman JavaScript - Array of Object, map(), push(), dan splice()

APLIKASI MANAJEMEN DATA MAHASISWA

Array of Object, map(), push(), dan splice()

DATA DIRI

Nama	: Latif Wibowo
-------------	----------------

NIM	: 0110224065
------------	--------------

Program Studi	: Teknik Informatika
----------------------	----------------------

Institusi	: STT Terpadu Nurul Fikri
------------------	---------------------------

1. Deskripsi Tugas

Pada Tugas Pertemuan 6 diminta membuat program manajemen data yang memisahkan data dan perintah ke dalam file berbeda. File data berisi 10 data berbentuk array of object, sedangkan file controller berisi tiga perintah: melihat, menambah, dan menghapus data.

Ketentuan dan implementasinya

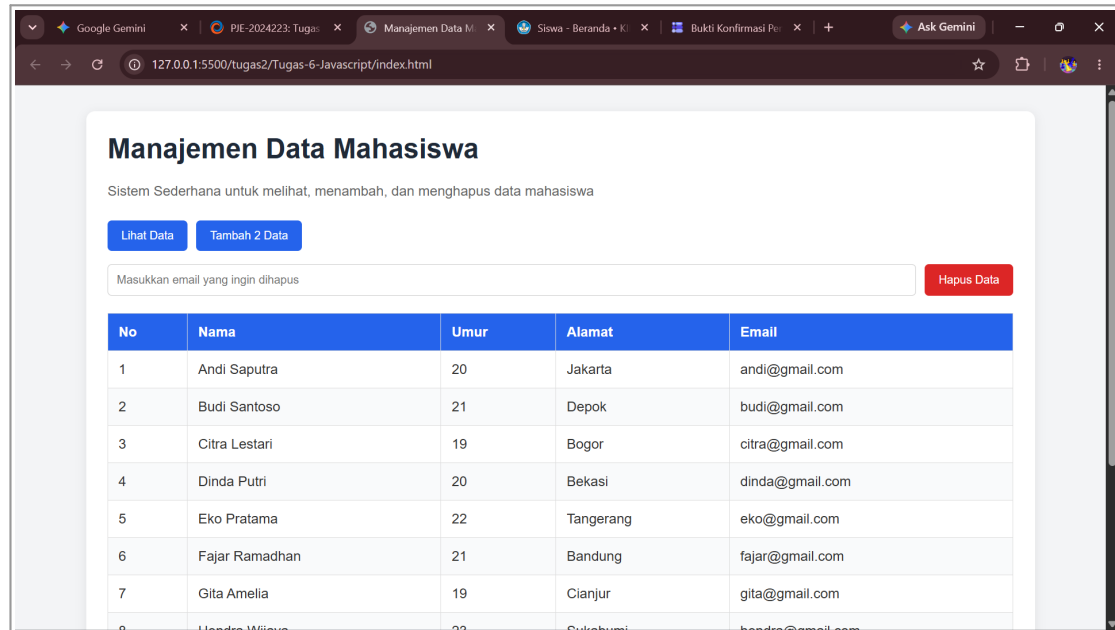
Ketentuan	Implementasi	Lokasi
10 data array of object (nama, umur, alamat, email)	Array dataMahasiswa berisi 10 mahasiswa	data.js
Controller dengan 3 perintah	lihatData(), tambahData(), hapusData()	controller.js
Tambah minimal 2 data dengan push	push() dua objek: Kiki Ananda dan Lina Marlina	controller.js
Tampilkan data dengan map()	dataMahasiswa.map(...) menjadi baris tabel	controller.js

Struktur File Proyek

File	Fungsi
index.html	Tombol aksi, input email, dan tabel data
data.js	Array dataMahasiswa berisi 10 data awal
controller.js	Fungsi lihat, tambah, dan hapus data
style.css	Tampilan halaman, tombol, dan tabel

2. Hasil Program

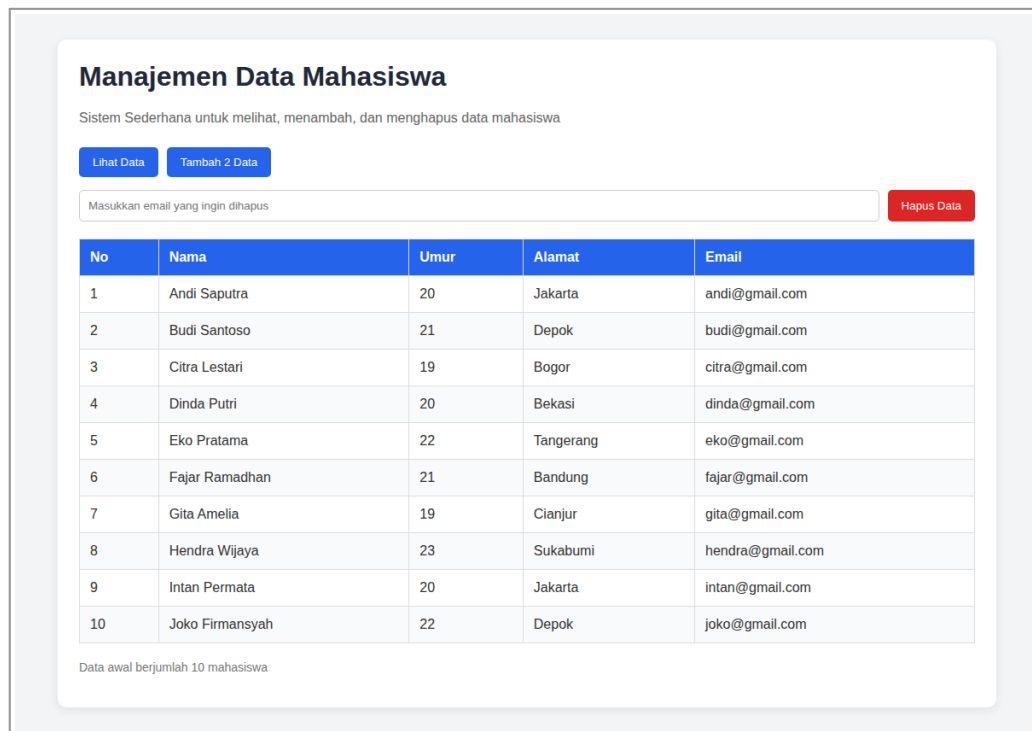
2.1 Tampilan Halaman di Browser



Gambar 1. Tampilan halaman index.html di browser (alamat 127.0.0.1:5500)

Halaman menampilkan judul **Manajemen Data Mahasiswa**, tombol **Lihat Data** dan **Tambah 2 Data**, kolom input email beserta tombol **Hapus Data** berwarna merah, serta tabel dengan kolom No, Nama, Umur, Alamat, dan Email.

2.2 Seluruh Data Awal (10 Data)



Gambar 2. Halaman dimuat penuh: 10 data awal tampil di tabel lewat map()

Kesepuluh data awal tampil dari Andi Saputra (nomor 1) sampai Joko Firmansyah (nomor 10), sesuai isi array di data.js.

2.3 Menambah 2 Data (push)

Manajemen Data Mahasiswa

Sistem Sederhana untuk melihat, menambah, dan menghapus data mahasiswa

Lihat DataTambah 2 Data

Hapus Data

No	Nama	Umur	Alamat	Email
1	Andi Saputra	20	Jakarta	andi@gmail.com
2	Budi Santoso	21	Depok	budi@gmail.com
3	Citra Lestari	19	Bogor	citra@gmail.com
4	Dinda Putri	20	Bekasi	dinda@gmail.com
5	Eko Pratama	22	Tangerang	eko@gmail.com
6	Fajar Ramadhan	21	Bandung	fajar@gmail.com
7	Gita Amelia	19	Cianjur	gita@gmail.com
8	Hendra Wijaya	23	Sukabumi	hendra@gmail.com
9	Intan Permata	20	Jakarta	intan@gmail.com
10	Joko Firmansyah	22	Depok	joko@gmail.com
11	Kiki Ananda	21	Bekasi	kiki@gmail.com
12	Lina Marlina	20	Bogor	lina@gmail.com

Data awal berjumlah 10 mahasiswa

Gambar 3. Setelah tombol Tambah 2 Data diklik, tabel menjadi 12 data

Klik tombol menjalankan **tambahData()**. Dua data baru, **Kiki Ananda** (nomor 11) dan **Lina Marlina** (nomor 12), muncul di akhir tabel dan alert "Berhasil menambahkan 2 data!" ditampilkan.

2.4 Menghapus Data berdasarkan Email

Manajemen Data Mahasiswa

Sistem Sederhana untuk melihat, menambah, dan menghapus data mahasiswa

Lihat DataTambah 2 Data

Hapus Data

No	Nama	Umur	Alamat	Email
1	Andi Saputra	20	Jakarta	andi@gmail.com
2	Budi Santoso	21	Depok	budi@gmail.com
3	Citra Lestari	19	Bogor	citra@gmail.com

Gambar 4. Email citra@gmail.com diketik pada kolom input sebelum tombol Hapus Data diklik

Manajemen Data Mahasiswa

Sistem Sederhana untuk melihat, menambah, dan menghapus data mahasiswa

Lihat Data

Tambah 2 Data

Hapus Data

No	Nama	Umur	Alamat	Email
1	Andi Saputra	20	Jakarta	andi@gmail.com
2	Budi Santoso	21	Depok	budi@gmail.com
3	Dinda Putri	20	Bekasi	dinda@gmail.com
4	Eko Pratama	22	Tangerang	eko@gmail.com
5	Fajar Ramadhan	21	Bandung	fajar@gmail.com
6	Gita Amelia	19	Cianjur	gita@gmail.com
7	Hendra Wijaya	23	Sukabumi	hendra@gmail.com
8	Intan Permata	20	Jakarta	intan@gmail.com
9	Joko Firmansyah	22	Depok	joko@gmail.com
10	Kiki Ananda	21	Bekasi	kiki@gmail.com
11	Lina Marlina	20	Bogor	lina@gmail.com

Data awal berjumlah 10 mahasiswa

Gambar 5. Setelah tombol Hapus Data diklik, data Citra Lestari hilang dan tabel menjadi 11 data

Fungsi **hapusData()** menemukan data dengan email tersebut lalu menghapusnya. Citra Lestari tidak lagi muncul, nomor urut tersusun ulang menjadi 1 sampai 11, dan alert "Data berhasil dihapus!" ditampilkan. Jika kolom email kosong, muncul alert "Silahkan masukkan email yang ingin dihapus!", dan jika email tidak ada di data, muncul alert "Data dengan email tersebut tidak ditemukan!".

3. Source Code Penting

Berikut potongan source code yang menjadi inti tugas, disertai penjelasan masing-masing.

3.1 Data Awal Mahasiswa (Array of Object)

```
1 // Data awal mahasiswa
2 const dataMahasiswa = [
3   {
4     nama: "Andi Saputra",
5     umur: 20,
6     alamat: "Jakarta",
7     email: "andi@gmail.com"
8   },
9   {
10    nama: "Budi Santoso",
11    umur: 21,
12    alamat: "Depok",
13    email: "budi@gmail.com"
14  },
15  {
16    nama: "Citra Lestari",
17    umur: 19,
18    alamat: "Bogor",
19    email: "citra@gmail.com"
20  },
```

Gambar 6. data.js (baris 1-20)

File **data.js** mendeklarasikan array **dataMahasiswa** yang berisi objek dengan empat properti sesuai soal: **nama**, **umur**, **alamat**, dan **email**. Capture ini menampilkan tiga data pertama; seluruhnya terdapat **10 data** (Andi Saputra sampai Joko Firmansyah) hingga baris 63.

3.2 Melihat Data dengan map()

```
1 // Elemen untuk menampilkan data
2 const tempatData = document.getElementById("tempatData");
3 const inputEmail = document.getElementById("emailHapus");
4
5 // Fungsi melihat data menggunakan map()
6 function lihatData() {
7     tempatData.innerHTML = dataMahasiswa.map((mahasiswa, index) => {
8         return `
9             <tr>
10                 <td>${index + 1}</td>
11                 <td>${mahasiswa.nama}</td>
12                 <td>${mahasiswa.umur}</td>
13                 <td>${mahasiswa.alamat}</td>
14                 <td>${mahasiswa.email}</td>
15             </tr>
16         `;
17     }).join("");
18 }
```

Gambar 7. controller.js (baris 1-18)

Fungsi **lihatData()** menampilkan data dengan **map()**. Method ini mengubah setiap objek mahasiswa menjadi baris tabel HTML (nomor urut, nama, umur, alamat, email) dalam template literal, lalu hasilnya digabung dengan `join("")` dan dimasukkan ke elemen `tbody#tempatData` lewat `innerHTML`.

3.3 Menambah Data dengan push()

```
20 // Fungsi menambah dua data menggunakan push()
21 function tambahData() {
22     dataMahasiswa.push(
23         {
24             nama: "Kiki Ananda",
25             umur: 21,
26             alamat: "Bekasi",
27             email: "kiki@gmail.com"
28         },
29         {
30             nama: "Lina Marlina",
31             umur: 20,
32             alamat: "Bogor",
33             email: "lina@gmail.com"
34         }
35     );
36     lihatData();
37     alert("Berhasil menambahkan 2 data!");
38 }
39 }
```

Gambar 8. controller.js (baris 20-39)

Fungsi **tambahData()** memanggil `push()` sekali dengan dua objek sekaligus, yaitu **Kiki Ananda** (Bekasi) dan **Lina Marlina** (Bogor), sehingga memenuhi ketentuan minimal 2 data. Setelah itu `lihatData()` dipanggil agar tabel langsung diperbarui dan alert "Berhasil menambahkan 2 data!" ditampilkan.

3.4 Menghapus Data dan Inisialisasi Tampilan

```
41 // Fungsi Menghapus data berdasarkan email
42 function hapusData() {
43     const email = inputEmail.value.trim();
44
45     if (email === "") {
46         alert("Silahkan masukkan email yang ingin dihapus!");
47         return;
48     }
49
50     const index = dataMahasiswa.findIndex(
51         mahasiswa => mahasiswa.email === email
52     );
53
54     if (index !== -1) {
55         dataMahasiswa.splice(index, 1);
56
57         lihatData();
58
59         alert("Data berhasil dihapus!");
60         inputEmail.value = "";
61     } else {
62         alert("Data dengan email tersebut tidak ditemukan!");
63     }
64 }
65
66 // Menampilkan data saat halaman pertama kali dimuat
67 lihatData();
```

Gambar 9. controller.js (baris 41-67)

Fungsi **hapusData()** mengambil email dari kolom input, lalu mencari posisinya dengan `findIndex()`. Jika ditemukan, data dihapus dengan `splice(index, 1)`, tabel diperbarui, dan muncul alert sukses. Ada tiga kondisi: email kosong (alert peringatan), email tidak ditemukan (alert "tidak ditemukan"), dan email ditemukan. Baris terakhir memanggil `lihatData()` agar data tampil saat halaman pertama dimuat.

3.5 Tombol Aksi dan Input Email di HTML

```
17     <div class="aksi">
18         <button onclick="lihatData()">
19             Lihat Data
20         </button>
21
22         <button onclick="tambahData()">
23             Tambah 2 Data
24         </button>
25     </div>
26
27     <div class="hapus">
28         <input
29             type="email"
30             id="emailHapus"
31             placeholder="Masukkan email yang ingin dihapus"
32         >
33
34         <button class="btn-hapus" onclick="hapusData()">
35             Hapus Data
36         </button>
37     </div>
```

Gambar 10. index.html (baris 17-37)

Tombol **Lihat Data** dan **Tambah 2 Data** memanggil fungsi lewat atribut `onclick`. Kolom input email (`#emailHapus`) dan tombol **Hapus Data** dipakai untuk menghapus data berdasarkan email.

3.6 Struktur Tabel dan Pemuatan Script

```
39     <div class="tabel-container">
40         <table>
41             <thead>
42                 <tr>
43                     <th>No</th>
44                     <th>Nama</th>
45                     <th>Umur</th>
46                     <th>Alamat</th>
47                     <th>Email</th>
48                 </tr>
49             </thead>
50             <tbody id="tempatData"></tbody>
51         </table>
52     </div>
53
54     <p class="catatan">
55         Data awal berjumlah 10 mahasiswa
56     </p>
57 </div>
58
59 <script src="data.js"></script>
60 <script src="controller.js"></script>
61 </body>
```

Gambar 11. index.html (baris 39-61)

Tabel memiliki thead berisi kolom No, Nama, Umur, Alamat, dan Email, sedangkan tbody ber-id tempatData diisi oleh JavaScript. File data.js dimuat sebelum controller.js agar variabel dataMahasiswa sudah tersedia saat controller dijalankan.

3.7 Styling Tombol dan Input

```
39 button {
40     padding: 10px 16px;
41     border: none;
42     border-radius: 5px;
43     background-color: #2563eb;
44     color: white;
45     cursor: pointer;
46 }
47
48 button:hover {
49     background-color: #1d4ed8;
50 }
51
52 input {
53     flex: 1;
54     min-width: 0;
55     padding: 10px;
56     border: 1px solid #ccc;
57     border-radius: 5px;
58 }
59
60 .btn-hapus {
61     background-color: #dc2626;
62 }
63
64 .btn-hapus:hover {
65     background-color: #b91c1c;
66 }
```

Gambar 12. style.css (baris 39-66)

CSS memberi tombol warna biru dengan efek hover, input yang melebar mengikuti sisa ruang (flex: 1), serta kelas `.btn-hapus` berwarna merah agar tombol hapus mudah dibedakan.

4. Kesimpulan

Program manajemen data mahasiswa berhasil dibuat sesuai petunjuk tugas. Data awal sebanyak 10 mahasiswa disimpan sebagai array of object di **data.js**, sedangkan **controller.js** berisi tiga perintah: **lihatData()** yang menampilkan data memakai **map()**, **tambahData()** yang menambah dua data memakai **push()**, dan **hapusData()** yang menghapus data berdasarkan email memakai **splice()**.